

ASCEND — IRoC-U 2026 UAV Project

STACK

PRE-ELIMINATION-ROUND	SUMMER ATTEMPT
PI-5	JETSON ORION NANO (8GB)
DOCKER	NATIVE UBUNTU 18
VINS-MONO	OPENVINS(BUILD AROUND UBUNTU 18)
OV9281	INTEL REALSENSE
PIXHAWK CUBE	PIXHAWK CUBE

Bottom line up front: the project has two parallel tracks that were never merged:

1. A **real hardware track** on a Raspberry-Pi-class or Jetson-class companion computer, running in Docker, with ROS1 Noetic, MAVROS, and either OpenVINS (`ov_msckf`) or VINS-Mono for VIO → this got as far as **arm/takeoff/hover/land with a vision-pose bridge**, plus a **calibration pipeline (Kalibr)** that was actually run at least twice.

1. Project Summary

- **Competition:** IRoC-U 2026, ASCEND challenge — Mars-analogue autonomous UAV survey.
- **Mission:** take off autonomously in a GPS-denied arena, survey terrain, capture HD imagery, downsample to 128×128 LR, match against seeded reference images, report matched-feature coordinates relative to a base station, return, land, transfer data, recharge, repeat if needed — with **zero human intervention** after start.
- **Success criteria (as designed):** full arena coverage, ≥1280×720 HD capture, >85% true-positive feature match rate, >80% false-rejection rate, >99% telemetry uptime, fully autonomous sortie.
- **Core design philosophy (as written in the technical manual):** determinism before performance, fail-safe at every layer (hardware/software/comms/power independently), simplicity compounds reliability.

2. TASKS

#	TASKS
1	Docking hardware, WiFi transfer, charging logic, base station software, ground telemetry display
2	YOLOv8n TensorRT, MobileNetV3 embeddings, ByteTrack, cosine-similarity matcher, reference seeding
3	MAVLink, ROS2 DDS, SiK radio, RTSP/GStreamer, topic architecture, data logging
4	VINS-Fusion/OpenVINS, D435i integration, EKF fusion, MAVROS bridge, geofence
5	Landing gear, docking alignment geometry, frame assembly, vibration isolation, thermal airflow
6	Propulsion calibration, ESC setup, battery management, pre-flight checks, firmware, ELRS pairing

4. Current System State at a Glance

Subsystem	Real status
Airframe, Pixhawk, motors, ESC, battery	⚠ Specified in BOM; assembly status not evidenced in notes — verify physically
ArduPilot firmware + parameters	⚠ Full param table specified (§6); which of these are actually flashed on the real Pixhawk is not confirmed in the notes — treat §6 as the target config to verify against the real vehicle
VIO (OpenVINS <code>ov_msckf</code> on real hardware)	✅ Real config files exist and were calibrated with Kalibr (real report dated 17-06-2026); real launch files exist for multiple variants
VIO (real-time reliability)	⚠ A separate "smooth IMU estimator" dead-reckoning fallback exists and was wired directly into <code>/mavros/vision_pose/pose</code> — this strongly implies real VIO fusion was not reliable enough to trust alone at some point (see §7.9)
Arm → OFFBOARD/GUIDED → hover → land	✅ Real, scripted, has explicit pre-flight checklist and topic-rate checks
Lawnmower survey	✅ Script exists (<code>takeoff_hover.py --mode survey ...</code>) and is invoked in the real runbook, but its source code is not

Subsystem	Real status
	included in any of these notes — find it on the Pi/Jetson filesystem or the <code>ISR0UC_2026</code> repo
Autonomous charging dock	 Fully designed (mechanical + RPi4 controller), no evidence of a built dock
ELRS-only telemetry redesign	 Proposal only
GCS / CLI / SDK / dashboard	 Proposal; a <code>gcs_dashboard.py</code> is invoked in the real runbook but its source is not in these notes

5. Hardware

5.1 Flight system (as specified — verify against the physical vehicle)

#	Component	Qty	Purpose
1	DJI F450 frame kit / TBS500	1	Airframe (450 mm wheelbase, target AUW $\leq$ 1.4 kg)
2	Pixhawk 2.4.8+ Power Module	1	Flight controller (STM32F427, MPU6000 + MPU9250, MS5611, HMC5983)
3	SunnySky X2212 980KV motors/ VELOX V2 MOTOR	4	Propulsion (~850 g thrust/motor @ 4S)
4	SpeedyBee 60A 4-in-1 ESC	1	DSHOT600 motor control
5	10×4.5 carbon propellers (CW+CCW)/8 INCH PROPS	2 pairs + 2 spare	Thrust
6	Tattu 4S 5600 mAh 50C LiPo	2	Primary + backup battery
7	XT60 connectors	4 pairs	Power
8	NVIDIA Jetson Orin Nano 8GB	1	Companion computer (6-core A78AE, 1024-core Ampere GPU, 2×NVDLA)
9	128 GB NVMe SSD	1	Jetson storage
10	Intel RealSense D435i	1	Global-shutter RGB+depth+IMU (BMI055, 200 Hz gyro / 400 Hz accel)
11	Holybro SiK 433 MHz radio (air)/RFD-900	1	Critical telemetry, $\leq$ 300 m LOS, 57600 baud MAVLink1
12	Holybro SiK 433 MHz radio (ground)/RFD-900	1	Ground telemetry

#	Component	Qty	Purpose
13	TP-Link Archer T3U Plus	1	5 GHz WiFi data link (RTSP + file transfer)
14	ELRS Nano receiver	1	Emergency manual RC override only (SBUS, <5 ms latency)
15	Matek UBEC 5V/10A	1	Dedicated Jetson power
16	FTDI USB-UART adapter	1	Jetson ↔ Pixhawk TELEM1
17	Powered USB 3.0 hub	1	USB expansion

5.3 Wiring (as specified)

BATTERY (4S LiPo, XT60)

└─> SPEEDYBEE 60A 4-IN-1 ESC → Motors 1-4 (PWM/DSHOT600) → Pixhawk MAIN OUT 1-4

└─> CUBE POWER MODULE → Pixhawk POWER port (V/I sense)

└─> ┌─5V BEC→ SiK radio, ELRS Rx, RealSense (via Jetson USB)

└─> MATEK UBEC 5V/10A → Jetson Orin Nano (barrel jack)

Pixhawk TELEM1 → FTDI → Jetson USB (MAVLink2, companion link)

Pixhawk TELEM2 → Holybro SiK radio (MAVLink1, ground link)

Pixhawk RC IN → ELRS Nano Rx (SBUS, manual override)

Motor layout (ArduPilot X-frame): 1=FR(CW), 2=BL(CCW), 3=FL(CCW)... wait see table below

Motor numbering (ArduPilot X, FRAME_CLASS=1 , FRAME_TYPE=1):

```

      FRONT
  1(CW)    4(CCW)
  [■]-----[■]
    |      +      |
  [■]-----[■]
  2(CCW)    3(CW)
      REAR

```

6. ArduPilot Configuration (target reference — verify on real Pixhawk before trusting)

6.1 Frame & motor

Param	Value	Notes
FRAME_CLASS	1	Quadcopter
FRAME_TYPE	1	X
MOT_PWM_TYPE	6	DSHOT600 (no ESC calibration needed)
MOT_SPIN_ARM	0.10	Min spin when armed
MOT_SPIN_MIN	0.15	Min spin in flight
MOT_THST_HOVER	0.35	Estimate — calibrate for real vehicle

6.2 Battery

Param	Value	Notes
BATT_MONITOR	4	Analog V+I
BATT_VOLT_PIN / BATT_CURR_PIN	2 / 3	Standard Pixhawk ADC pins
BATT_VOLT_MULT	10.1	Calibrate for your actual power module
BATT_AMP_PERVLT	17.0	Same
BATT_LOW_VOLT / BATT_CRT_VOLT	14.4 / 14.0	3.6V/cell warn, 3.5V/cell critical (4S)
BATT_LOW_MAH	800	Low remaining-capacity warning
BATT_FS_LOW_ACT	2	RTL
BATT_FS_CRT_ACT	1	Land immediately

Cross-check from `running.md` Q&A: real battery config confirmed as **4S LiPo, ~16.8V full, 14.0V cutoff** — consistent with the table above.

6.3 RC / GCS failsafe

Param	Value	Notes
FS_THR_ENABLE / FS_THR_VALUE / FS_THR_ACT	1 / 975 / 1	RTL on throttle failsafe
FS_GCS_ENABLE / FS_GCS_TIMEOUT / FS_GCS_ACT	1 / 5 / 2	Continue autonomous mission if GCS heartbeat lost (companion computer loss is handled separately, see §15)

6.4 Geofence

Param	Value
FENCE_ENABLE	1
FENCE_TYPE	3 (circle + altitude)
FENCE_RADIUS	15 m (set to arena size)
FENCE_ALT_MAX	8 m
FENCE_ACTION	1 (RTL)
FENCE_MARGIN	1.0 m

6.5 EKF3 / External Nav (VIO input) — this is the block that matters most day to day

Param	Value	Notes
EK3_ENABLE	1	
EK3_SRC1_POSXY	6	External nav (VIO)
EK3_SRC1_POSZ	1 (manual doc) — but real runbook sets this to 6 as well	See conflict note below
EK3_SRC1_VELXY	6	External nav
EK3_SRC1_VELZ	6	External nav
EK3_SRC1_YAW	1	Compass
GPS_TYPE	0	GPS disabled (GPS-denied arena)
AHRS_EKF_TYPE	3	
VISO_TYPE	1	Enable visual odometry input
VISO_POS_X/Y/Z	0.05 / 0.0 / -0.02	D435i offset from FC — measure on the real airframe, don't trust this number blindly
VISO_YAW	0	
VISO_DELAY_MS	50	Expected VIO latency

Conflict to resolve: the technical manual (IRS0-Sys-Arch.md) lists `EK3_SRC1_POSZ = 1` (barometer for altitude), but the actual commands run on real hardware (ISR0UC-2026.md , Terminal 4) set **all four** of `EK3_SRC1_POSXY/POSZ/VELXY/VELZ = 6` — i.e. altitude too comes from VIO on the real system, not baro. Also seen on the real system: `GPS1_TYPE = 0` and `ARMING_CHECK = 0` (arming checks disabled — **this is a safety-relevant deviation**,

understand why before flying, it's likely there to allow arming without GPS/compass pre-arm checks in a GPS-denied bench/indoor test).

6.6 Position controller

Param	Value
WPNAV_SPEED	50 cm/s (0.5 m/s survey speed)
WPNAV_SPEED_UP / WPNAV_SPEED_DN	100 / 75 cm/s
WPNAV_ACCEL	50 cm/s ²
LOIT_SPEED / LOIT_ACC_MAX	200 cm/s / 100 cm/s ²
LAND_SPEED / LAND_SPEED_HIGH	30 / 75 cm/s

6.7 PID starting point (run AutoTune before trusting these)

Param	Value
ATC_RAT_RLL_P/I/D	0.135 / 0.135 / 0.0036
ATC_RAT_PIT_P/I/D	0.135 / 0.135 / 0.0036
ATC_RAT_YAW_P/I/D	0.180 / 0.018 / 0.000

AutoTune procedure: set RC7_OPTION=17 , take off in AltHold/Loiter, flip switch high, fly ~5–10 min while ArduPilot excites each axis, flip low to save, land/disarm and confirm params saved.

6.8 Flight mode mapping

RC5 position	Mode
Low (PWM<1300)	Stabilize (manual recovery)
Mid (1300–1700)	Loiter (pre-mission check)
High (>1700)	Auto (mission execution)

The companion computer commands the vehicle in **Guided/OFFBOARD** mode via MAVROS during autonomous operation; the RC switch is the human override.

7. Real Hardware Software Stack — THE PART THAT ACTUALLY FLIES

This is the Docker/ROS1-Noetic stack. Two overlapping presentations of it exist in the notes (ISROUC-2026.md = raw session log, ALL_in_one.md = cleaned-up packaged version). Below is the merged, de-duplicated version.

7.1 Repo & container layout

```
ISROUC_2026/
├─ Dockerfile
├─ docker-compose.yml
├─ entrypoint.sh
├─ scripts/
│   └─ start_stack.sh    # boots roscore + MAVROS + camera + VINS
│   └─ mission.sh        # takeoff → hover → land
```

Inside the container (catkin workspace referenced throughout): /catkin_ws/ — contains devel/setup.bash, imu_camera_sync.py, camera_node.py, vins_to_mavros.py, vins_monitor.py, gcs_dashboard.py, takeoff_hover.py. **None of these script bodies are present in the source notes** except the ones reproduced below (vision_pose_bridge.py, simple_pose_bridge.py, imu_smooth_estimator.py, start_stack.sh, mission.sh). Locate the rest on the actual device/repo.

The other real workspace referenced in CONFIGS.md is /home/uasorin/uav_ws/ (OpenVINS-based, not VINS-Mono) — see §7.5. Confirm whether /catkin_ws (Docker) and /home/uasorin/uav_ws (bare-metal) are the **same** logical workspace mounted two ways, or genuinely two different builds. Evidence suggests OpenVINS (ov_msckf) is the more recently maintained VIO engine (it has the most launch-file variants and a dated Kalibr report), while VINS-Mono appears in the earlier Docker/Pi packaging.

7.2 Docker run command (real, full — Raspberry-Pi-class device with rpikam / libcamera)

```
export DRONE_IP=$(ip route get 8.8.8.8 | awk '{print $7; exit}')

docker run -it --rm \
  --name vins_mono \
  --net=host --privileged \
  --device /dev/video0 --device /dev/video1 \
  --device /dev/media0 --device /dev/media1 \
  --device /dev/ttyACM0 --device /dev/ttyUSB0 \
  --volume /run/udev:/run/udev:ro \
  --volume /run/libcamera:/run/libcamera:ro \
  --volume /dev/shm:/dev/shm \
  --volume /usr/lib/aarch64-linux-gnu/libcamera.so.0.7.0:/usr/lib/aarch64-
```

```

linux-gnu/libcamera.so.0.7.0:ro \
  --volume /usr/lib/aarch64-linux-gnu/libcamera.so.0.7:/usr/lib/aarch64-linux-
gnu/libcamera.so.0.7:ro \
  --volume /usr/lib/aarch64-linux-gnu/libcamera.so:/usr/lib/aarch64-linux-
gnu/libcamera.so:ro \
  --volume /usr/lib/aarch64-linux-gnu/libcamera-
base.so.0.7.0:/usr/lib/aarch64-linux-gnu/libcamera-base.so.0.7.0:ro \
  --volume /usr/lib/aarch64-linux-gnu/libcamera-base.so.0.7:/usr/lib/aarch64-
linux-gnu/libcamera-base.so.0.7:ro \
  --volume /usr/lib/aarch64-linux-gnu/libcamera-base.so:/usr/lib/aarch64-
linux-gnu/libcamera-base.so:ro \
  --volume /usr/lib/aarch64-linux-gnu/libcamera:/usr/lib/aarch64-linux-
gnu/libcamera:ro \
  --volume /usr/share/libcamera:/usr/share/libcamera:ro \
  --volume /usr/share/rpicam-apps:/usr/share/rpicam-apps:ro \
  --volume /usr/bin/rpicam-vid:/usr/bin/rpicam-vid:ro \
  --volume /usr/lib/aarch64-linux-
gnu/librpicam_app.so.1.11.1:/usr/lib/aarch64-linux-
gnu/librpicam_app.so.1.11.1:ro \
  --volume /usr/lib/aarch64-linux-gnu/librpicam_app.so.1:/usr/lib/aarch64-
linux-gnu/librpicam_app.so.1:ro \
  --volume /usr/lib/aarch64-linux-gnu/librpicam_app.so:/usr/lib/aarch64-linux-
gnu/librpicam_app.so:ro \
  --volume $(pwd)/scripts:/scripts \
  -e ROS_MASTER_URI="http://${DRONE_IP}:11311" \
  -e ROS_IP="${DRONE_IP}" \
  -e ROS_HOSTNAME="${DRONE_IP}" \
  isro26:latest bash

```

Persistent container lifecycle (don't recreate every time):

```

docker stop vins_mono          # stop
docker start -i vins_mono      # restart, attached
docker exec -it vins_mono bash # open another shell into the same running
container

```

7.3 entrypoint.sh (auto-detects LAN IP so you stop hardcoding it)

```

#!/bin/bash
set -e
source /opt/ros/noetic/setup.bash

if [ -f /catkin_ws/devel/setup.bash ]; then
  source /catkin_ws/devel/setup.bash
  echo "[ISRO26] Workspace sourced"

```

```

else
    echo "[ISR026] Workspace not built"
fi

if [ -z "$ROS_IP" ] || [ "$ROS_IP" = "127.0.0.1" ]; then
    LAN_IP=$(ip route get 8.8.8.8 2>/dev/null | awk '{print $7; exit}')
    export ROS_MASTER_URI="http://${LAN_IP}:11311"
    export ROS_IP="$LAN_IP"
    export ROS_HOSTNAME="$LAN_IP"
    echo "[ISR026] ROS_IP auto-set to $LAN_IP"
fi

service chrony start 2>/dev/null || true
echo "[ISR026] ROS Noetic | EGO-Swarm | VINS-Mono | MAVROS"
echo "[ISR026] ROS_MASTER_URI=$ROS_MASTER_URI"
exec "$@"

```

chrony is started for clock sync — this matters because VIO/EKF timestamp drift between the companion computer and the Cube is a known open question (see §15, "VIO-to-EKF timestamp drift").

7.4 IMU–Camera calibration procedure (Kalibr) — real, was executed at least twice

Run before anything else — IMU_SYNC sequence:

1. Start `roscore`.
2. Confirm `/camera/image_raw` is publishing.
3. Confirm `/mavros/imu/data` is publishing.
4. `rostopic list` — sanity check.
5. `python3 imu_camera_sync.py` — monitor params live.
6. `python3 imu_camera_sync.py --record --duration 60` — records the calibration bag.

During recording — mandatory motion pattern (calibration fails without this):

- Figure-8 motion.
- Rotate in roll, pitch, **and** yaw.
- Translate forward/back and up/down.
- **Avoid:** pure straight-line motion, or holding still.

Expected output: `/catkin_ws/calib.bag`, `/catkin_ws/imu_params.yaml`, a printed `td: X.XXXXXX` (camera-IMU time offset).

Run on your laptop (not necessarily inside Docker):

```
kalibr_calibrate_imu_camera \  
  --bag /catkin_ws/calib.bag \  
  --cam /catkin_ws/ov9281_calibration.yaml \  
  --imu /catkin_ws/imu_params.yaml \  
  --target /catkin_ws/april_6x6.yaml
```

Confirmed real calibration run — dated 17-06-2026:

```
Report written to /home/uasorin/kalibration-17-06-2026/calib_motion-report-  
cam.pdf  
Report written to /home/uasorin/kalibration-17-06-2026/calib_motion-report-  
imucam.pdf
```

After Kalibr completes, update `ov_msckf/config/d435i_stereo.yaml` fields: `cam0_f/c/d` and `cam1_f/c/d` (intrinsic from `camchain-imucam.yaml`), `T_C0toI / T_C1toI` (invert `T_ci` if needed), `gyro/accel noise + random walk` (from `imu.yaml`), and `calib_camimu_dt` (from `timeshift_cam_imu`). Same values also need mirroring into `kalibr_imucam_chain.yaml`, `kalibr_imu_chain.yaml`, `imu0.yaml`, and `cam0.yaml` since these are loaded via `relative_config_imu / relative_config_imucam`.

Camera/RC host-side capture (Pi camera, when not using RealSense):

```
rpicam-vid --width 640 --height 480 --framerate 30 \  
  --codec mjpeg --timeout 0 \  
  --output /dev/shm/cam.mjpeg --npreview &
```

Always run `pkill -f rpicam || true` before restarting this; check with `ps aux | grep rpicam`; stop with `killall rpicam-vid`.

7.5 OpenVINS (`ov_msckf`) — real, calibrated configuration values

Real file: `home/uasorin/uav_ws/src/ov_msckf/config/d435i_stereo.yaml` (the "primary target" per the notes).

Topics & stereo:

```
cam0_topic: "/cam0/image_raw"  
cam1_topic: "/cam1/image_raw"  
imu_topic:  "/camera/imu"  
use_stereo: true  
max_cameras: 2
```

IMU noise (D435i datasheet values, later overwritten by Kalibr/imu_utils results):

```
gyroscope_noise_density: 0.000698
gyroscope_random_walk: 0.0000873
accelerometer_noise_density: 0.003924
accelerometer_random_walk: 0.000196
calib_camimu_dt: 0.004818 # from Kalibr
```

Estimator tuning (as calibrated):

```
max_clones: 11
max_slam: 50
max_slam_in_update: 25
max_msckf_in_update: 40
dt_slam_delay: 1
use_klt: true
num_pts: 200
fast_threshold: 30
min_px_dist: 15
grid_x: 5
grid_y: 5
knn_ratio: 0.70
track_frequency: 20.0
downsample_cameras: false
num_opencv_threads: 4
histogram_method: "HISTOGRAM"
init_window_time: 2.0
init_imu_thresh: 1.5
init_max_disparity: 10.0
init_max_features: 75
try_zupt: true
zupt_max_velocity: 0.05
zupt_max_disparity: 0.35
zupt_noise_multiplier: 2.0
use_fej: true
use_imuavg: true
use_rk4int: true
gravity_mag: 9.81
feat_rep_msckf: "GLOBAL_3D"
feat_rep_slam: "ANCHORED_MSCKF_INVERSE_DEPTH"
calib_cam_extrinsics: false
calib_cam_intrinsics: false
calib_cam_timeoffset: false
use_aruco: false
```

Camera intrinsics/extrinsics from Kalibr (real, calibrated values — cam0 = infra1, cam1 = infra2):

```
cam0_wh: [848, 480]
cam0_f: [433.4210719702908, 433.1126853219418]
cam0_c: [419.43358998728337, 244.45719868418513]
cam0_d: [0.0028087014699872317, -0.0010112835231954698, 7.422082893059053e-05,
-0.003548923367109757]
cam0_distortion_model: "radtan"
T_C0toI:
- [0.99997936, 0.00199618, -0.00610745, -0.00953317]
- [-0.00197538, 0.99999224, 0.00340855, -0.00102937]
- [0.00611420, -0.00339641, 0.99997554, 0.02284829]
- [0.0, 0.0, 0.0, 1.0]

cam1_wh: [848, 480]
cam1_f: [433.16475184434427, 433.1886168855474]
cam1_c: [418.7411791453085, 244.42499356427626]
cam1_d: [0.0018546817098996251, 8.350689524137114e-05,
-0.00015533016181284016, -0.0035210591243086207]
cam1_distortion_model: "radtan"
T_C1toI:
- [0.99998336, 0.00197450, -0.00542048, 0.04119092]
- [-0.00195721, 0.99999298, 0.00319428, -0.00109201]
- [0.00542675, -0.00318362, 0.99998021, 0.02286689]
- [0.0, 0.0, 0.0, 1.0]
```

Baseline check: T_C1tol x-offset (0.0412 m) minus T_C0tol x-offset (-0.0095 m) $\approx$ **5.07 cm stereo baseline** — matches the D435i's nominal ~50 mm IR baseline. Good sanity check if you recalibrate.

`ov_msckf/config/*.yaml` **files that exist in the real workspace** (multiple parallel variants — be careful which one a launch file actually loads): `ascend_launch.yaml`, `cam0.yaml`, `d435i_stereo.yaml` (+ `.bak`), `d435i_stereo/estimator_config.yaml`, `d435i_stereo/kalibr_imucam_chain.yaml` (+ `.bak`), `d435i_stereo/kalibr_imu_chain.yaml`, `imu0.yaml`, `ov_msckf_default.yaml`, `ov_msckf_vio.yaml`, `your_config.yaml`.

There are also two simplified/example configs used elsewhere (`ascend_launch/config/opencvins_d435i.yaml`, `opencvins_d455.yaml`) with **placeholder intrinsics** (381.5, 381.5, 323.7, 238.5) that must not be flown with — replace with the real Kalibr values above before using either.

7.6 Real `.launch` files (ROS1) — inventory & what each does

Launch file	MAVROS	Camera driver
<code>full_stack.launch</code>	—	—
<code>full_stack_fixed.launch</code>	<code>fcu_url=/dev/ttyACM0:921600</code>	<code>realsense2_camera_nodelet</code> reduced FPS (15), manual exposure/gain params
<code>vio_stack.launch</code>	Included separately (<code>mavros/px4.launch</code> , <code>/dev/ttyACM0:921600</code>)	<code>rs_camera.launch</code> , gyro 200/accel 100 Hz, infra 640×480@30
<code>uav_openvins.launch</code>	<code>fcu_url=/dev/ttyACM0:921600</code> + <code>params/mavros.yaml</code>	Same <code>rs_camera</code> pattern
<code>working_stack.launch</code>	<code>mavros_node</code> only	<code>rs_camera</code> , 30 fps everywhere
<code>complete_system.launch</code>	Args <code>fcu_url</code> , <code>enable_vio</code>	<code>rs_camera</code> 30fps

`params/mavros.yaml` (real, minimal):

```
vision_pose: {listen: true}
```

7.7 Vision-pose bridge scripts (VIO → MAVROS, includes the ENU→NED-style axis fix)

`vision_pose_bridge.py` — swaps X/Y and negates Z (this is the frame correction between the VIO output frame and what ArduPilot expects), also broadcasts a TF:

```
#!/usr/bin/env python3
import rospy, tf
from nav_msgs.msg import Odometry
from geometry_msgs.msg import PoseStamped

class Bridge:
    def __init__(self):
        rospy.init_node('vision_pose_bridge')
        self.pub = rospy.Publisher('/mavros/vision_pose/pose', PoseStamped,
queue_size=10)
        rospy.Subscriber('/ov_msckf/odomimu', Odometry, self.cb)
        self.br = tf.TransformBroadcaster()
```

```

def cb(self, odom):
    pose = PoseStamped()
    pose.header = odom.header
    pose.header.frame_id = "map"
    pose.pose.position.x = odom.pose.pose.position.y
    pose.pose.position.y = odom.pose.pose.position.x
    pose.pose.position.z = -odom.pose.pose.position.z
    pose.pose.orientation = odom.pose.pose.orientation
    self.pub.publish(pose)
    self.br.sendTransform((pose.pose.position.x, pose.pose.position.y,
pose.pose.position.z),
                           (pose.pose.orientation.x,
pose.pose.orientation.y,
                           pose.pose.orientation.z,
pose.pose.orientation.w),
                           odom.header.stamp, "vision_origin", "map")

if __name__ == '__main__':
    Bridge(); rospy.spin()

```

simple_pose_bridge.py — no axis correction, passes pose straight through (use only if your VIO output frame already matches ArduPilot's expected frame — **do not mix these up**, using the wrong one will silently corrupt the EKF fusion):

```

#!/usr/bin/env python3
import rospy
from geometry_msgs.msg import PoseStamped
from nav_msgs.msg import Odometry

class SimplePoseBridge:
    def __init__(self):
        rospy.init_node('simple_pose_bridge')
        self.pub = rospy.Publisher('/mavros/vision_pose/pose', PoseStamped,
queue_size=10)
        rospy.Subscriber('/ov_msckf/odomimu', Odometry, self.cb)
    def cb(self, msg):
        pose = PoseStamped()
        pose.header = msg.header
        pose.pose = msg.pose.pose
        self.pub.publish(pose)

if __name__ == '__main__':
    SimplePoseBridge(); rospy.spin()

```

7.8 Mission scripts — real, packaged version (ALL_in_one.md)

scripts/start_stack.sh — boots roscore → MAVROS (auto-detects /dev/ttyACM0 or /dev/ttyUSB0) → camera (auto-detects pixel format via v4l2-ctl) → VINS-Mono (vins_estimator + loop_fusion, euroc config):

```
#!/bin/bash
set -e
source /opt/ros/noetic/setup.bash
source /catkin_ws/devel/setup.bash

roscore &
sleep 4

if [ -c /dev/ttyACM0 ]; then FCU="/dev/ttyACM0:921600"
elif [ -c /dev/ttyUSB0 ]; then FCU="/dev/ttyUSB0:921600"
else echo "[ERROR] No Pixhawk device found"; exit 1; fi
roslaunch mavros px4.launch fcu_url:="$FCU" &
sleep 5

FMT=$(v4l2-ctl --device=/dev/video0 --list-formats-ext 2>/dev/null \
    | grep -oiP "(YUYV|MJPEG|GREY|Y10)" | head -1 | tr '[:upper:]'
    '[:lower:]')
FMT=${FMT:-yuyv}
ROS_NAMESPACE=cam0 rosrun usb_cam usb_cam_node \
    _video_device:=/dev/video0 _image_width:=640 _image_height:=400 \
    _pixel_format:="$FMT" _framerate:=60 _camera_frame_id:=cam0
_io_method:=mmap &
sleep 3

CONFIG="/catkin_ws/src/VINS-Mono/config/euroc/euroc_config.yaml"
roslaunch vins_estimator eureka.launch config_file:="$CONFIG" &
sleep 3
roslaunch loop_fusion loop_fusion.launch config_file:="$CONFIG" &
wait
```

scripts/mission.sh — takeoff → hover(N min) → land, with the OFFBOARD pre-streaming step ArduPilot requires:

```
#!/bin/bash
HOVER_MINUTES=2
HOVER_ALTITUDE=2.0    # metres

# 1. Wait for FCU connection (rostopic echo /mavros/state, connected: True)
# 2. Pre-stream setpoints at 20 Hz for 3s BEFORE switching mode -
```

```
PX4/ArduPilot
#   refuses OFFBOARD unless setpoints are already streaming.
rostopic pub -r 20 /mavros/setpoint_position/local geometry_msgs/PoseStamped \
  "header: {frame_id: 'map'}
    pose: {position: {x: 0.0, y: 0.0, z: ${HOVER_ALTITUDE}}, orientation: {w:
1.0}}" &
SETPOINT_PID=$!
sleep 3

# 3. Switch OFFBOARD, 4. Arm, 5. Hover for HOVER_SECS with countdown,
# 6. kill setpoint stream, switch AUTO.LAND
rosservice call /mavros/set_mode "custom_mode: 'OFFBOARD'"
rosservice call /mavros/cmd/arming "value: true"
# ... hover loop ...
kill $SETPOINT_PID
rosservice call /mavros/set_mode "custom_mode: 'AUTO.LAND'"
```

Deploy procedure (real, on the Pi):

```
git clone https://github.com/hardsubjectParth/ISROUC_2026.git
cd ISROUC_2026
mkdir -p scripts
# paste start_stack.sh / mission.sh, then:
chmod +x scripts/start_stack.sh scripts/mission.sh
git add . && git commit -m "add mission and stack launch scripts" && git push
```

Every flight:

- Terminal 1 (host): `docker run ... ($7.2) bash /scripts/start_stack.sh`
- Terminal 2: `docker exec -it vins_mono bash → bash /scripts/mission.sh`
- Laptop (optional): RViz, export `ROS_MASTER_URI=http://<PI_IP>:11311`, add
`/vins_estimator/odometry`, `/mavros/local_position/pose`, `/mavros/state`

Separately, the "final mission" invocation style (`ISROUC-2026.md`) uses a different script name and CLI flags:

```
python3 /catkin_ws/takeoff_hover.py --mode hover --alt 1.5 --secs 15

python3 /catkin_ws/takeoff_hover.py --mode survey \
  --alt 2.0 --length 10 --width 8 --rows 4 --speed 1.0 # lawnmower 10×8m, 4
rows
```

This script's source is **not present** in the notes — `mission.sh` above is a bash-equivalent reimplementation of the hover/land part only; the survey mode logic must be recovered from the device or the repo.

7.9 ⚠ The dead-reckoning "FAKE VIO" fallback — read this carefully

The real session notes literally header this section **"FAKE"**. It replaces real VIO output with a pure IMU double-integration ("smooth IMU estimator") published straight onto `/mavros/vision_pose/pose`. This exists because manual/visual flying needed *some* position feed into the EKF and the real VIO pipeline apparently wasn't the thing being trusted at that point in testing.

Flow:

```
/mavros/imu/data → imu_smooth_estimator.py (low-pass + damped double
integration)
                        → /mavros/vision_pose/pose → MAVROS EKF3 →
/mavros/local_position/pose → RViz
```

`imu_smooth_estimator.py` (real, as run):

```
#!/usr/bin/env python3
import rospy
from sensor_msgs.msg import Imu
from geometry_msgs.msg import PoseStamped
import numpy as np

pos = np.zeros(3); vel = np.zeros(3)
VEL_DAMP = 0.92      # lower = more stable / slower response
POS_DAMP = 0.997     # closer to 1 = less drift suppression
alpha = 0.2          # low-pass filter coefficient; lower = smoother but
                      # laggy
acc_filtered = np.zeros(3)
pub = None

def imu_cb(msg):
    global pos, vel, acc_filtered
    dt = 0.02 # assumes ~50Hz
    acc = np.array([msg.linear_acceleration.x, msg.linear_acceleration.y,
                    msg.linear_acceleration.z + 9.81])
    acc_filtered = alpha * acc + (1 - alpha) * acc_filtered
    vel[:] = VEL_DAMP * (vel + acc_filtered * dt)
    pos[:] = POS_DAMP * (pos + vel * dt)
```

```

    ps = PoseStamped()
    ps.header.stamp = rospy.Time.now(); ps.header.frame_id = "map"
    ps.pose.position.x, ps.pose.position.y, ps.pose.position.z = map(float,
pos)
    ps.pose.orientation = msg.orientation
    pub.publish(ps)

rospy.init_node("imu_smooth_estimator")
pub = rospy.Publisher("/mavros/vision_pose/pose", PoseStamped, queue_size=10)
rospy.Subscriber("/mavros/imu/data", Imu, imu_cb)
rospy.spin()

```

Paired EKF params (applied live via `mavparam`, real):

```

roslaunch mavros mavparam set EK3_SRC1_POSXY 6
roslaunch mavros mavparam set EK3_SRC1_POSZ 6
roslaunch mavros mavparam set EK3_SRC1_VELXY 6
roslaunch mavros mavparam set EK3_SRC1_VELZ 6
roslaunch mavros mavparam set GPS1_TYPE 0
roslaunch mavros mavparam set ARMING_CHECK 0
roslaunch mavros mavsys rate --all 50

```

Design-review Q&A on this exact fallback (from `running.md`) — capture these decisions:

- Max safe **dead-reckoning hover duration before drift becomes a collision risk: 5 seconds**, based on the Cube's IMU gyro drift rates.
- Failsafe policy: *"If VIO fails mid-flight → hover using dead reckoning within the Cube's drift limits; if not recovered, land; if recovered, continue mission."*
- Emergency landing: *"if RealSense is available, use its depth map to find a flat spot; otherwise use a 1D LiDAR on the bottom of the drone to land straight down safely."*
- Innovation-gating ("flyaway gate") was flagged as an **open question** — no concrete EKF innovation-variance threshold had been decided at time of writing.

→ **Action item for you:** confirm whether this fallback is still the production path or whether real VIO (§7.5) has since become trustworthy enough to remove it. Don't delete this script without understanding why it was added.

7.10 Manual multi-terminal runbook (real IPs, real sequence — useful for debugging when the scripted path fails)

Real static IP seen throughout: 10.195.249.82 (Pi/ROS master), example laptop 10.46.61.40 / 10.46.61.82 in an earlier session. **Always** `unset ROS_MASTER_URI`

ROS_HOSTNAME ROS_IP before re-exporting — stale env vars were clearly a recurring problem.

Terminal 1 (Pi) – roscore:

```
docker start -ai vins_mono
unset ROS_MASTER_URI ROS_HOSTNAME ROS_IP
export ROS_MASTER_URI=http://<PI_IP>:11311
export ROS_HOSTNAME=<PI_IP>
source /opt/ros/noetic/setup.bash
roscore
# wait for "started core service [/rosout]" before continuing
```

Terminal 2 – MAVROS:

```
docker exec -it vins_mono bash
<same env exports>
source /catkin_ws/devel/setup.bash
roslaunch mavros apm.launch fcu_url:=/dev/ttyACM0:921600
# wait for "FCU: ArduCopter" line
```

Terminal 2B – set MAVLink rate:

```
roslaunch mavros mavsys rate --all 50
```

Terminal 3 – IMU estimator (§7.9) OR real VIO node (§7.5), not both:

```
python3 /catkin_ws/imu_smooth_estimator.py
```

Terminal 4 – EKF params (run once, Ctrl+C after): see §7.9 mavparam block

Terminal 5 – TF broadcaster (static map→odom→base_link, only needed without a real TF tree):

```
roslaunch tf2_ros static_transform_publisher 0 0 0 0 0 0 1 map odom &
roslaunch tf2_ros static_transform_publisher 0 0 0 0 0 0 1 odom base_link &
```

Terminal 6 – verify:

```
rostopic hz /mavros/imu/data          # expect ~200Hz
rostopic hz /mavros/vision_pose/pose  # expect ~50Hz
rostopic hz /drone/trajectory          # expect ~50Hz
```

7.11 RViz on laptop (real)

```
xhost +local:docker
docker run -it --rm --name rviz_client --net=host \
  -e DISPLAY=$DISPLAY \
  -e ROS_MASTER_URI=http://<PI_IP>:11311 \
  -e ROS_IP=$(hostname -I | awk '{print $1}') \
  -v /tmp/.X11-unix:/tmp/.X11-unix \
  vins_mono bash
```

```
# inside container:
source /opt/ros/noetic/setup.bash
rviz
```

Setup: Fixed Frame = `map` ; Add Path → `/drone/trajectory` (green, line width 0.03–0.05);
Add Pose → `/mavros/local_position/pose` (red arrow); save with `File` → `Save Config As`
→ `~/drone.rviz`, relaunch later with `rviz -d ~/drone.rviz`.

7.12 Pre-flight checklist & expected topic rates (real, run before every mission)

```
rostopic echo -n1 /mavros/state          # connected: True, armed: False
rostopic echo -n1 /mavros/imu/data        # ~100–200 Hz
rostopic echo -n1 /cam0/usb_cam/image_raw # streaming
rostopic hz /mavros/setpoint_position/local # ~20 Hz during mission

# Once flying:
rostopic hz /camera/image_raw             # ~28 Hz
rostopic hz /mavros/imu/data               # ~200 Hz
rostopic hz /vins_estimator/odometry       # ~10 Hz
rostopic hz /mavros/vision_pose/pose       # ~10 Hz
rostopic echo /mavros/state                # armed/mode/connected
rostopic echo /vins_estimator/odometry | grep -A3 position
```

If `connected: False` on `/mavros/state`, MAVROS isn't talking to the Pixhawk yet — everything else depends on this being `True` first. Wait, or check the FCU device path (`/dev/ttyACM0` vs `/dev/ttyUSB0`).

8. Simulation Stack — Gazebo Harmonic + ArduPilot SITL + ROS2 Humble (WSL2)

This is a **separate environment**, used to rehearse navigation logic (Nav2, OctoMap, EGO-Planner) without risking the real airframe. It does not talk to the real Pixhawk — ArduPilot SITL simulates the flight controller in software.

8.1 Why it exists

- De-risk Nav2/OctoMap/lawnmower logic before flying.
- Test at 640×480/15fps to fit in 16 GB RAM on WSL2.

8.2 Install (condensed — see `complete_system.md` for the full script)

Known WSL2 apt issue and fix (run **first**, every fresh WSL install):

```
sudo rm -rf /var/lib/apt/lists/*
sudo apt clean && sudo apt autoclean
sudo dpkg --configure -a
sudo sed -i 's|http://security.ubuntu.com|http://archive.ubuntu.com|g'
/etc/apt/sources.list
sudo apt update && sudo apt upgrade -y --fix-missing
```

Then, in order: system base tools → ROS2 Humble (desktop) → colcon/rosdep/vcstool → Gazebo Harmonic (gz-harmonic) → ros-humble-ros-gz* bridge → MAVROS (+ GeographicLib datasets script) → vision/perception deps (cv-bridge, PCL, OpenCV) → ArduPilot SITL (./waf configure --board sitl && ./waf copter) → Python MAVLink tools (pymavlink , MAVProxy , dronekit) → ROS2 workspace (~/ascend_ws) → VINS-Fusion → EGO-Planner v2 → remaining ROS2 packages (rviz2, rosbag2, xacro) → SQLite/ZMQ tools → environment variables in ~/.bashrc (GZ_SIM_RESOURCE_PATH , FCU_URL=udp://:14550@ , ROS_DOMAIN_ID=0 , LIBGL_ALWAYS_SOFTWARE=0).

Sanity check script (~/check_stack.sh) verifies: ros2 --version , gz sim --version , ArduCopter SITL binary, OpenCV import, MAVROS package list, workspace install dir.

Fallback if apt hash mismatch persists:

```
sudo apt install -y --allow-unauthenticated ros-humble-desktop # option A
# or download+dpkg -i manually # option B
sudo rm -rf /var/lib/apt/lists/* && sleep 300 && sudo apt update && sudo apt
upgrade -y # option C
```

8.3 World / drone model (real files, run.md)

- drone_description/worlds/vio_world.sdf — Gazebo Harmonic world with obstacles for VIO feature tracking.
- drone_description/models/vio_drone/model.sdf + model.config — the simulated quadrotor with stereo cameras + IMU.
- A second, simpler world exists in ISRO-SIM.md: ~/IROC2026/worlds/OpenVINS.sdf — flat ground + 2 walls + 1 box obstacle, ODE physics, max_step_size=0.004.

8.4 Bridges & config (real)

- drone_bringup/config/bridge.yaml — ROS2↔Gazebo topic bridge definitions (gz_topic@gz_msg_type[dir]ros_topic:ros_msg_type).

- `openvins_config/config/estimator_config.yaml` — WSL2-optimized (640×480, 15fps, reduced feature count) OpenVINS config with stereo baseline 0.12 m, histogram equalization for indoor lighting.
- `drone_bringup/config/nav2.yaml` — lightweight Nav2 config tuned for an aerial vehicle (not a ground robot — controller/costmap tuning differs from stock Nav2 examples).
- `drone_bringup/config/mavros_params.yaml` , `drone_bringup/config/rviz.rviz` .

8.5 Launch order (6+ terminals, or `scripts/launch_all.sh` which opens labeled GNOME terminals automatically)

```

1. Gazebo world          gz sim
~/.ascend_ws/src/ascend_world/worlds/ascend_world.sdf
2. ArduPilot SITL        sim_vehicle.py -v ArduCopter -f gazebo-iris --
console --map
3. MAVROS                ros2 launch mavros apm.launch fcu_url:=udp://:14550@
4. ROS↔Gazebo bridge     ros2 run ros_gz_bridge parameter_bridge <topic list>
5. RViz2                rviz2
6. VINS-Fusion / OpenVINS ros2 launch vins vins_rviz.launch.py
config_file:=...
7. (ISRO-SIM variant) OctoMap ros2 run octomap_server octomap_server_node --
ros-args -p resolution:=0.15
8. (ISRO-SIM variant) Mission + Telemetry python nodes

```

Expected topics after bring-up: `/mavros/state` , `/mavros/local_position/pose` , `/mavros/imu/data` , `/camera/color/image_raw` , `/camera/depth/image_raw` , `/scan` , `/tf` , `/estimator/odometry` (or `/ov_msckf/*` for OpenVINS variant).

The `ISRO-SIM.md` variant additionally includes a **lawnmower mission node** (`mission_node.py` , hardcoded 6-waypoint pattern, pure `time.sleep` timing — a placeholder/demo, not closed-loop) and a **telemetry node** (`telemetry_node.py` , subscribes `/mavros/local_position/pose` , prints altitude).

WSL2-specific tuning: camera 640×480@15fps, OctoMap resolution 0.15 m, avoid Gazebo Fuel assets/high-poly meshes/shadows if laggy, only open RViz displays you need.

8.6 Troubleshooting (real, from `run.md` — keep this table, it will save you hours)

Symptom	Fix
Gazebo won't start / black screen on WSL2	<code>export LIBGL_ALWAYS_SOFTWARE=1; export MESA_GL_VERSION_OVERRIDE=3.3</code> or <code>export GZ_SIM_RENDERER=ogre2</code>

Symptom	Fix
Bridge topics not appearing in ROS2	<code>ros2 node info /gz_ros_bridge; gz topic -l \ grep imu; pkill -f parameter_bridge && ros2 launch drone_bringup bridge.launch.py</code>
MAVROS not connecting to SITL	<code>netstat -tulnp \ grep 14550; test with mavproxy.py --master=udp:127.0.0.1:14550; check ros2 param get /mavros fcu_url matches SITL's output port</code>
OpenVINS fails to initialize	Check <code>ros2 topic hz /stereo/left/image_raw</code> (~15 Hz) and <code>/imu/data_raw</code> (~200 Hz); verify <code>use_sim_time</code> is true everywhere; increase world lighting or lower <code>init_imu_thresh</code>
OctoMap empty / not updating	Check <code>/depth/points</code> rate; verify <code>frame_id</code> matches odom frame; restart via <code>scripts/restart_octomap.sh</code>
Nav2 lifecycle nodes not activating	<code>ros2 lifecycle get /planner_server /controller_server; manually configure then activate; or scripts/restart_nav2.sh</code>
High CPU/RAM on WSL2	Lower <code>real_time_update_rate</code> in world SDF to 200; reduce OpenVINS <code>num_pts</code> to 50 and <code>max_clones</code> to 6; <code>pkill -f rviz2; check with top -p \$(pgrep -d',' -f ros2)</code>
Clock sync / sim_time issues	<code>ros2 param set /mission_node use_sim_time true; check /clock</code> is publishing; <code>sim_time</code> should already be forced in the provided launch files

Quick reference:

```
cd ~/drone_ws
colcon build --symlink-install --cmake-args -DCMAKE_BUILD_TYPE=Release --parallel-workers 4
source install/setup.bash
~/drone_ws/scripts/launch_all.sh      # launch everything
~/drone_ws/scripts/kill_all.sh       # stop everything
colcon build --packages-select drone_mission --symlink-install  # rebuild one package
~/drone_ws/scripts/check_topics.sh
ros2 run tf2_tools view_frames && evince frames.pdf
```

9. Mission Logic / State Machine

9.1 Designed FSM (12 states, IRS0-Sys-Arch.md §12) — 
designed, only partially implemented

State	Entry	Exit
BOOT	Power on	All services running
MANUAL_SAFE	BOOT complete	Operator arms
TAKEOFF	Armed + Guided	Altitude reached ± 0.2 m
VIO_INIT	Post-takeoff	VIO covariance < 0.05
SURVEY	VIO initialized	All waypoints visited OR battery low
RETURN_HOME	Survey complete OR battery $< 25\%$	Home reached
LAND	Return-home complete	Disarmed (contact detected)
DATA_TRANSFER	Landed + WiFi connected	All files transferred
CHARGE	Data transfer complete	SoC $\geq 90\%$
VALIDATE	Charging complete	Validation result received
COMPLETE	All features validated	—
NEXT_SORTIE / ABORT	Validation failed & sorties remain / any critical fault	— / manual reset

What actually exists today: TAKEOFF $\rightarrow$ hover(N min) $\rightarrow$ LAND (§7.8, real) and a standalone lawnmower survey mode flag on `takeoff_hover.py` (real invocation, source not recovered). VIO_INIT, DATA_TRANSFER, CHARGE, VALIDATE have **no evidenced implementation** — they exist only in the design manual and the `mission_fsm_node.py` skeleton reproduced there (ROS2, not wired to real hardware).

9.2 Mission flow diagram (from ISRO-DESGIN.canvas) — diagram only, no code

Node flow (paraphrased from the canvas boxes): UPLOAD FILE (seed images) $\rightarrow$ `sortie_No == 0` $\rightarrow$ MISSION COMMAND $\rightarrow$ START MISSION $\rightarrow$ communication up $\rightarrow$ MOTOR CHECK $\rightarrow$ ARM $\rightarrow$ VIO static time $\rightarrow$ VIO READY $\rightarrow$ JETSON SENDS READY $\rightarrow$ TAKE OFF $\rightarrow$ GO STRAIGHT $\rightarrow$ branch on IF YELLOW encountered / NOT encountered $\rightarrow$ YAW 90 (both branches) $\rightarrow$ Detect features $\rightarrow$ get all features $\rightarrow$ TIME STAMP + LOCAL COORDINATES $\rightarrow$ Spatial awareness $\rightarrow$ branch IF good / IF BAD $\rightarrow$ LR-conversion $\rightarrow$ save LR $\rightarrow$ HR images of features saved, VIDEO in .bag file $\rightarrow$ MEMORY-DUMP $\rightarrow$ TRANSFER Folder $\rightarrow$ base station receives IMAGE-SEED + IMAGE-RGB.

This is a useful **logical reference for how the mission was intended to branch** (colour-based obstacle/marker cue "YELLOW", spatial-awareness good/bad gating before saving) but

there's no corresponding code — use it as a spec when someone eventually builds the mission FSM + CV pipeline for real.

10. Computer Vision Pipeline — DESIGNED, NOT IMPLEMENTED

Fully specified in `IRS0-Sys-Arch.md §10`, no evidence of execution anywhere else in the notes.

- **Detector:** YOLOv8n, exported to TensorRT FP16 (`scripts/export_yolo_trt.py` → `config/trt_engines/yolov8n_fp16.engine`), targeting the Jetson GPU.
- **Embedding model:** MobileNetV3, exported to ONNX then to a DLA-core TensorRT engine (`trtexec --onnx=... --useDLACore=0 --allowGPUPFallback --fp16`).
- **Tracking:** ByteTrack.
- **Matching:** cosine similarity between live embeddings and pre-seeded reference embeddings (`scripts/capture_references.py` , run before competition).
- **Threshold tuning:** `scripts/tune_threshold.py` , ROC-curve based, run on Day 10 with 50 matching + 50 non-matching pairs in arena lighting.
- **Image pipeline:** HD capture ($\geq 1280 \times 720$) → 128×128 LR area-interpolation conversion, with identical CLAHE applied to both reference and live captures (critical — mismatched preprocessing breaks matching).
- **CPU/accelerator budget:** YOLO on GPU, MobileNetV3 on DLA core 0, ByteTrack + cosine on CPU cores 4–5, VIO reserved on cores 0–1 (never share).
- **Thermal governor:** $>70^{\circ}\text{C}$ → drop YOLO to 5 fps; $>80^{\circ}\text{C}$ → pause CV entirely, VIO continues (CV is explicitly *not* safety-critical).

To build this from scratch: start with the YOLO export script, verify TensorRT engine runs standalone on the Jetson, then wire ByteTrack, then the embedding matcher, in that order — that's the dependency order the design implies.

11. Telemetry & Comms

11.1 Actual (what's really wired)

Dual channel today: MAVLink over the companion-computer USB link (TELEM1 → FTDI → Jetson/Pi, real) + SiK radio (TELEM2, 433 MHz, ground link) + WiFi/ROS network for RViz and topic inspection (static IPs, `ROS_MASTER_URI` / `ROS_IP` / `ROS_HOSTNAME` triplet set on every

terminal — this triplet mismatch was clearly a recurring source of connection failures, always `unset` before re-exporting).

12. Ground Control / CLI / SDK / GCS Dashboard

- **What's real:** a `gcs_dashboard.py` is **invoked** in the real runbook (`python3 gcs_dashboard.py` inside the container, with `ROS_MASTER_URI / ROS_IP` set) — its source is **not included** in any of these notes. It exists somewhere on the device/repo; find and document it separately.
 - **What's designed but not built:** `IRS0-Sys-Arch.md §17` and `IRS0-IDEAS.md` both describe a much larger **ASCEND CLI/SDK** — command groups for mission control, telemetry dashboard, drone management, CV tooling, a deterministic **replay system** (flagged as the most important feature — replay logged flights deterministically for debugging), logging/data management, validation, deployment, profiling, arena mapping, docking/charging commands, diagnostics, recovery commands, config system, and simulation support. This is a full ground-station product pitch — none of it appears implemented. If you build it, the replay system is called out explicitly as the highest-value feature to prioritize.
-

13. Autonomous Charging System — **DESIGNED, NOT BUILT**

- Docking station: precision funnel guide rails (mechanical, `IRS0-Sys-Arch.md §17`), spring-loaded brass contacts (4×, 5N ±3mm), WS2812B LED ring for alignment feedback, ArUco marker (20cm, ID=0) for precision landing.
 - Controller: Raspberry Pi 4 running `charging_controller.py` (contact detection → charge initiation → SoC monitoring via INA219 → charge-complete signal).
 - **Contingency already decided:** if charging integration is failing by Day 17 of the build schedule, the team competes with **manual battery swaps** instead — the mission FSM must support a manual-swap mode as a backup path. This is a real prioritization decision (§18/Priority Stack), not just a nice-to-have.
-

15. Known Issues, Errors & Fixes — Consolidated

#	Issue	Where seen	Fix / m
1	WSL2 apt hash-sum mismatch on <code>ros-humble-desktop</code> install	<code>complete_system.md</code>	Clear <code>dpkg --get-selections</code> archive update missin unauth downlo and ret
2	Stale <code>ROS_MASTER_URI</code> / <code>ROS_IP</code> / <code>ROS_HOSTNAME</code> env vars causing silent connection failures across Docker terminals	<code>ISROUC-2026.md</code>	Always <code>ROS_HOSTNAME</code> exportin
3	MAVROS never finds the Pixhawk device path	<code>ALL_in_one.md</code> , <code>run.md</code>	Auto-de then <code>/dev</code> verify S <code>-tuln</code> with ma master
4	ArduPilot refuses OFFBOARD/GUIDED mode switch	<code>ALL_in_one.md</code>	Must p for $\geq 3s$ this is a require
5	<code>connected: False</code> on <code>/mavros/state</code> blocks everything downstream	<code>ISROUC-2026.md</code>	MAVRO True b topic; if device/
6	RealSense pixel format / exposure instability corrupting VIO feature tracking	<code>CONFIGS.md</code> (<code>full_stack_fixed.launch</code>)	Lock e: (<code>lock_ module explicit (stere gain=1 stability</code>
7	Wrong pose-bridge script used (axis-swap vs pass-through) silently corrupts EKF fusion	§7.7 above	Confirm (<code>visio simple your VI before</code>
8	Real VIO not trusted enough at some point in testing	<code>ISROUC-2026.md</code> "FAKE" section	Dead-r (<code>imu_s substiti</code>

#	Issue	Where seen	Fix / m
			/mavrc workan
9	VIO-to-EKF timestamp drift between companion computer and Cube	running.md (open design question)	chrony entryp (e.g. M an ope
10	No decided EKF innovation-variance threshold for a "flyaway gate" (VIO says moved 5m, IMU says 0.1m)	running.md (open design question)	Unresc before flight
11	Placeholder camera intrinsics (381.5, 381.5, 323.7, 238.5) left in openvins_d435i.yaml / openvins_d455.yaml	CONFIGS.md	Replac calibrat using e
12	Gazebo black screen / won't render on WSL2	run.md	LIBGL_ MESA_C or switc GZ_SIM
13	OpenVINS fails to initialize in sim	run.md	Check and IM expect: use_si lighting
14	High CPU/RAM on WSL2 during full stack	run.md	Lower real_t reduce max_cl rviz2
15	Nav2 lifecycle nodes stuck unconfigured	run.md	Manua /planr activa restai
16	ArUco/YOLO/charging pipeline entirely unimplemented but scheduled	elimination_round.md risk register	See §1 largest

16. Failure Modes & Contingency (as designed — cross-check against what's actually wired today, since only the hover/land path is confirmed real)

Fault	Immediate action	Recovery	Prevention
VIO loss during flight	HOLD (loiter) via MAVROS	Re-init VIO ×3 attempts @5s each	If all fail: autonomous RTL on baro+compass only; keep VIO pinned to cores 0–1, nothing else runs there
Battery critical (<20% SoC)	Immediate RTL, land regardless of mission state	—	Conservative 25% SoC early-return warning
CV pipeline stall (thermal)	Drop YOLO to 5fps; VIO unaffected	30s cooldown, resume at reduced fps	FP16 TensorRT, active airflow, monitor tegrastats
Charging contact failure	Log failure, request human verification	Manual battery swap (breaks full-autonomy requirement — avoid)	Test 20+ consecutive landings before relying on it
WiFi loss mid-mission	Continue autonomously, no abort	Auto-reconnect every 30s, transfer data on reconnect	Dedicated 5GHz AP, clear LOS
Feature validation failure (0 matches after full survey)	RTL, transfer, recharge, re-seed with adjusted threshold	Lower cosine threshold by 0.05 on sortie 2; escalate after 3 failed sorties	Validate reference-embedding quality before Day 15
MAVROS heartbeat lost >3s (companion computer crash)	Pixhawk auto-RTL via FS_GCS failsafe	Jetson watchdog attempts node restart (30s)	FS_GCS_ENABLE=1 , FS_GCS_TIMEOUT=5 , FS_GCS_ACT=1
Geofence breach	ArduPilot hardware-level auto-RTL	—	FENCE_ENABLE=1 , sized to arena

Non-negotiable rules carried over from the program plan (still good practice regardless of schedule):

1. Gate/milestone failures stop the line — fix before integrating further.
2. VIO is sacred: nothing else runs on CPU cores 0–1, ever, during flight.
3. Every test flight is logged (ROS2/rosbag + GCS log + written note) — no undocumented flights.

4. Weight budget is measured, not assumed — don't fly overweight.
 5. Never start a sortie below 90% battery SoC.
 6. The pre-flight checklist (§7.12) is mandatory, every flight, no exceptions.
-

17. RiskS

Risk
VIO drift on featureless terrain, position error >1m
Battery flight time <10 min, insufficient for full survey
TensorRT conversion fails for MobileNetV3 on DLA
Charging contact fails to align reliably
Cosine-similarity threshold produces false matches
Thermal throttling degrades VIO during sustained flight
ROS2 DDS discovery latency causes missed detections
Arena lighting washes out D435i IR
WiFi congestion at venue
Frame vibration corrupts IMU
ELRS/SiK frequency conflict

Command Cheat-Sheet

```
# Container lifecycle
docker stop vins_mono
docker start -i vins_mono
docker exec -it vins_mono bash

# Real flight (packaged scripts)
bash /scripts/start_stack.sh      # roscore + MAVROS + camera + VIO
bash /scripts/mission.sh         # takeoff -> hover -> land

# Pre-flight checks
rostopic echo -n1 /mavros/state
rostopic hz /mavros/imu/data
rostopic hz /mavros/setpoint_position/local
```

```
# Calibration
```

```
python3 imu_camera_sync.py --record --duration 60
```

```
kalibr_calibrate_imu_camera --bag calib.bag --cam ov9281_calibration.yaml \  
  --imu imu_params.yaml --target april_6x6.yaml
```

```
# Simulation stack
```

```
gz sim ~/ascend_ws/src/ascend_world/worlds/ascend_world.sdf
```

```
sim_vehicle.py -v ArduCopter -f gazebo-iris --console --map
```

```
ros2 launch mavros apm.launch fcu_url:=udp://:14550@
```

```
ros2 launch vins vins_rviz.launch.py config_file:=<path>
```

```
~/drone_ws/scripts/launch_all.sh
```

```
~/drone_ws/scripts/kill_all.sh
```
